

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..34

OVERVIEW

 Copy as Markdown

Overview

A future call is a piece of server work you schedule to run later instead of right now. You pick a method, say when it should run, and Serverpod invokes it at that time, even if the request that scheduled it has long since finished. Common uses are a welcome email an hour after sign-up, a reminder a day before an appointment, or a cleanup job that runs every night.

Future calls are stored in the database, so they survive a server restart and are shared across every server instance you run. This makes them different from an in-memory **Timer**: a scheduled call is not lost if the process that created it stops.

What you can schedule

You write a future call as a method on a class, generate the type-safe code, and then schedule it in one of three ways:

- **After a delay**, for example one hour from now.
- **At a specific time**, for example a fixed date and time in the future.
- **On a repeating schedule**, either a fixed interval (every 20 minutes) or a [cron](#) expression (every day at 02:00 UTC). Cron is a standard text format for describing repeating schedules.

Execution guarantees

A scheduled call runs **at least once**. Across all your running server instances, one instance claims each call and normally runs it a single time. If that instance crashes partway through, the call is picked up and run again by another instance, so a call can occasionally run more than once. Write work that is safe to repeat (for example, check whether the email was already sent before sending it).

If the method itself throws an exception, Serverpod logs the error and does not retry the call. You are responsible for retrying failed work: schedule a new call if the work needs to happen. Recurring calls



Ask AI

exception, since the next run is always scheduled regardless of whether the current one succeeds.

Where future calls run

Which servers run future calls depends on their role, the mode a server is started in. Servers in the default **monolith** role run them, and future calls need a database. A server started in the **serverless** role does not run them. If you host in a serverless environment, run a separate scheduled process in the **maintenance** role to execute due calls. See [server roles](#) and [hosting elsewhere](#).

What each page covers

- **Future calls:** define a future call, schedule it after a delay or at a time, and cancel scheduled calls.
- **Recurring tasks:** run a call on a repeating cron or interval schedule.
- **Inheritance:** extend future calls from other classes and modules.
- **Configuration:** set concurrency, the scan interval, and broken-call handling.

Related

- [Sessions](#): the session a future call receives when it runs.
- [Server roles](#): which roles run scheduled work.
- [Configuration reference](#): every future-call key with its environment variable.